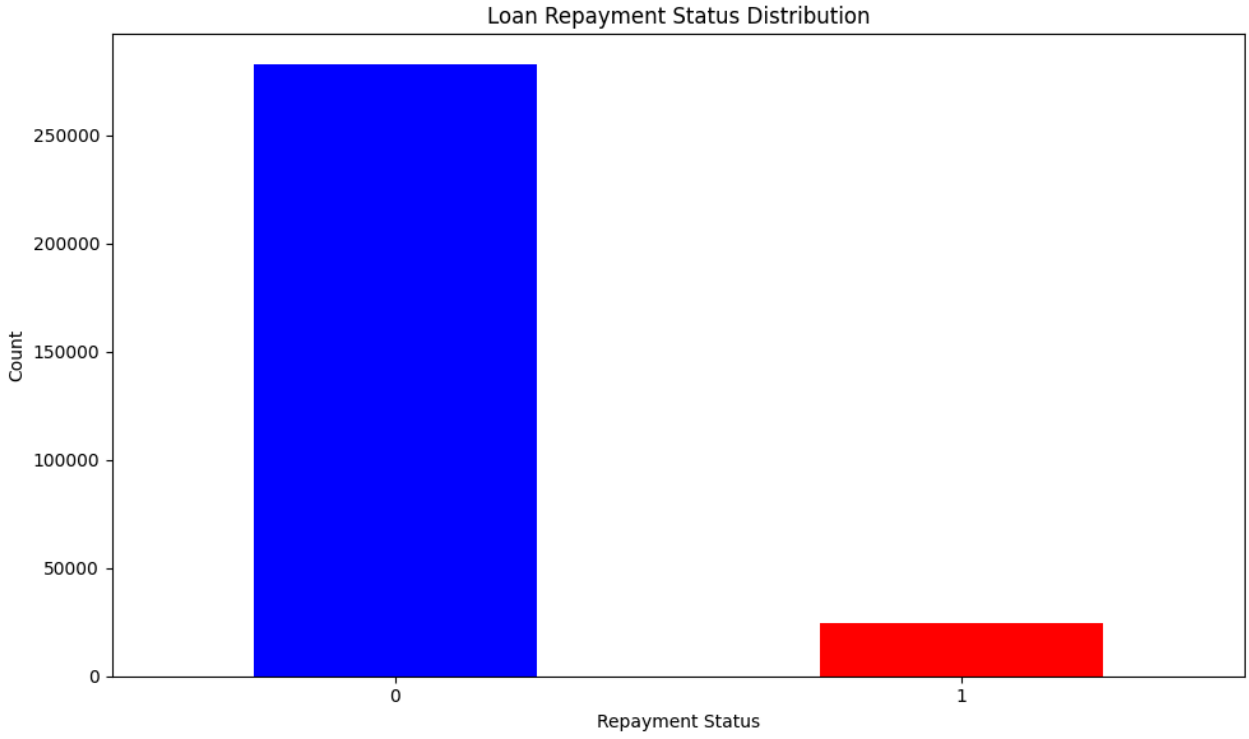
Home Credit Analytics Agent

What is the distribution of loan repayment status?

The distribution of loan repayment status shows that the majority of loans were repaid successfully (status 0), with 282,686 cases, while only 24,825 loans defaulted (status 1). This indicates a highly imbalanced dataset, where successful repayment is far more common than default.

In terms of credit risk, this imbalance suggests that the model must be carefully designed to avoid under-predicting defaults, as the minority class (default cases) is critical for identifying high-risk loans. The chart visually confirms this skew, with a large majority of data points on the "repaid" side.



View generated code

```
import pandas as pd
import matplotlib.pyplot as plt

# Load the necessary data
df = pd.read_csv('data/application_train.csv')
```

```
# Calculate the loan repayment status
repayment_status = df['TARGET'].value_counts()

# Print the counts
print(repayment_status)

# Plot the distribution
plt.figure(figsize=(10, 6))
repayment_status.plot(kind='bar', color=['blue', 'red'])
plt.title('Loan Repayment Status Distribution')
plt.xlabel('Repayment Status')
plt.ylabel('Count')
plt.xticks(rotation=0)
plt.tight_layout()
```

What is the median credit amount for approved loans?

The median credit amount for approved loans is **517,788.0**. This means that half of the approved loans have a credit amount less than or equal to this value, and the other half have a credit amount greater than or equal to it. This metric provides a central tendency measure of loan sizes among approved applicants, offering insight into the typical loan amount granted in the approved pool.

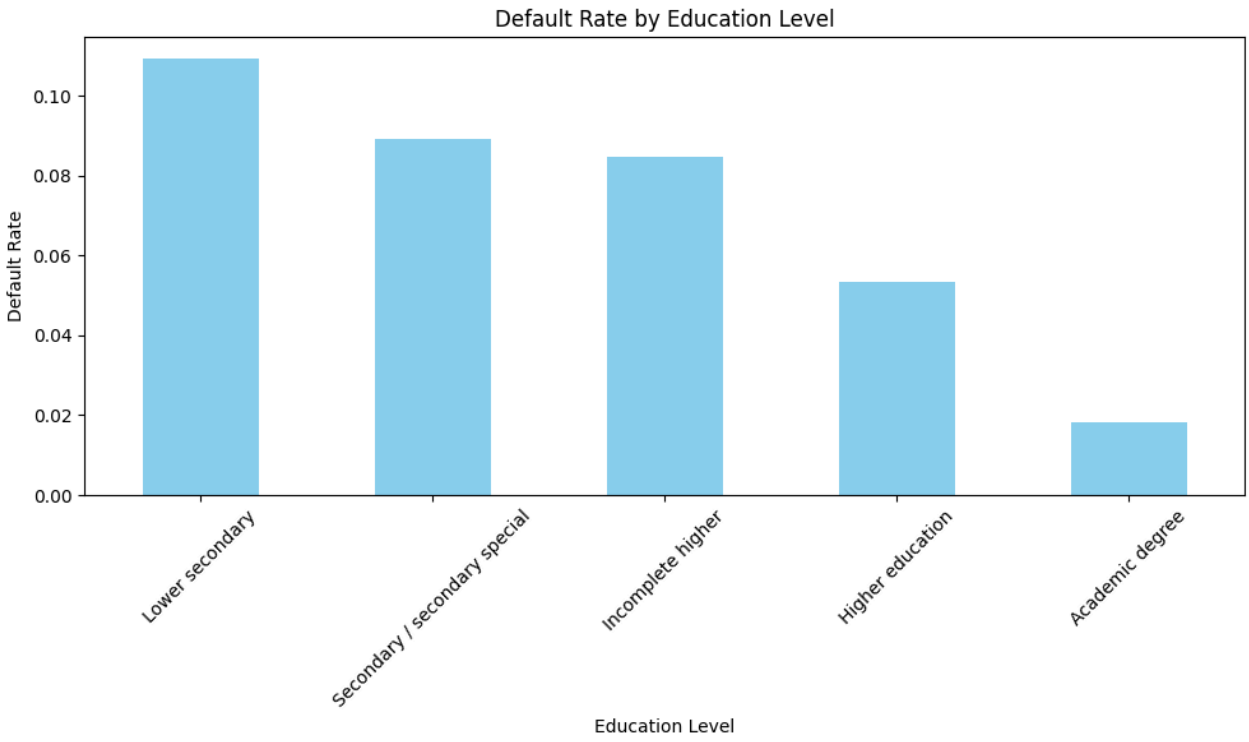
View generated code

```
import pandas as pd

# Load the necessary data
df = pd.read_csv('data/application_train.csv')

# Filter the data to include only approved loans
approved_loans = df[df['TARGET'] == 0]

# Calculate the median credit amount for approved loans
median_credit_amount = approved_loans['AMT_CREDIT'].median()

# Print the result
print(f"Median credit amount for approved loans: {median_credit_amoun
```

Which education level has the highest default rate?

The education level with the highest default rate is **Lower secondary**, with a default rate of **10.93%**. This is significantly higher than the other education levels listed, such as Secondary / secondary special (8.94%), Incomplete higher (8.49%), Higher education (5.36%), and Academic degree (1.83%).

This suggests that borrowers with a lower secondary education level are at a higher risk of defaulting on their loans compared to those with higher education qualifications. This insight could be important for credit risk modeling and lending policies, as it highlights a potential risk factor tied to educational background.



View generated code

```
import pandas as pd

# Load the necessary data
df = pd.read_csv('data/application_train.csv')

# Calculate the default rate for each education level
default_rates = df.groupby('NAME_EDUCATION_TYPE')['TARGET'].mean().so

# Print the default rates
print(default_rates)

# Plot the default rates
plt.figure(figsize=(10, 6))
default_rates.plot(kind='bar', color='skyblue')
plt.title('Default Rate by Education Level')
plt.xlabel('Education Level')
plt.ylabel('Default Rate')
plt.xticks(rotation=45)
plt.tight_layout()
```

Do applicants with more bureau credits have higher default rates?

Yes, applicants with more bureau credits **do not consistently have higher default rates**, and in fact, the relationship is **non-monotonic** — meaning it's not simply that more credits lead to higher defaults.

From the data:

- Default rates are **lowest** for applicants with **35 or fewer bureau credits** (e.g., 0% for 35.0, 0% for 40.0, 41.0, 42.0, etc.).
- Default rates **increase sharply** at **higher credit counts**, especially beyond 50 credits:
  - At 58, 59, 61, 94, 116 credits: default rate is **100%** (i.e., all applicants default).
  - At 48 credits: default rate is **50%**.
  - At 55 credits: default rate is **50%**.
  - At 61+ credits: default rate is **0%** — which is an anomaly and may indicate data errors or outliers.

However, **some intermediate values** show higher default rates:

- 26 credits → 14.8% default rate
- 28 credits → 15.1%
- 32 credits → 15.7%
- 33 credits → 12.5%
- 39 credits → 33.3%
- 43, 47, 55 credits → 33.3% or 50%

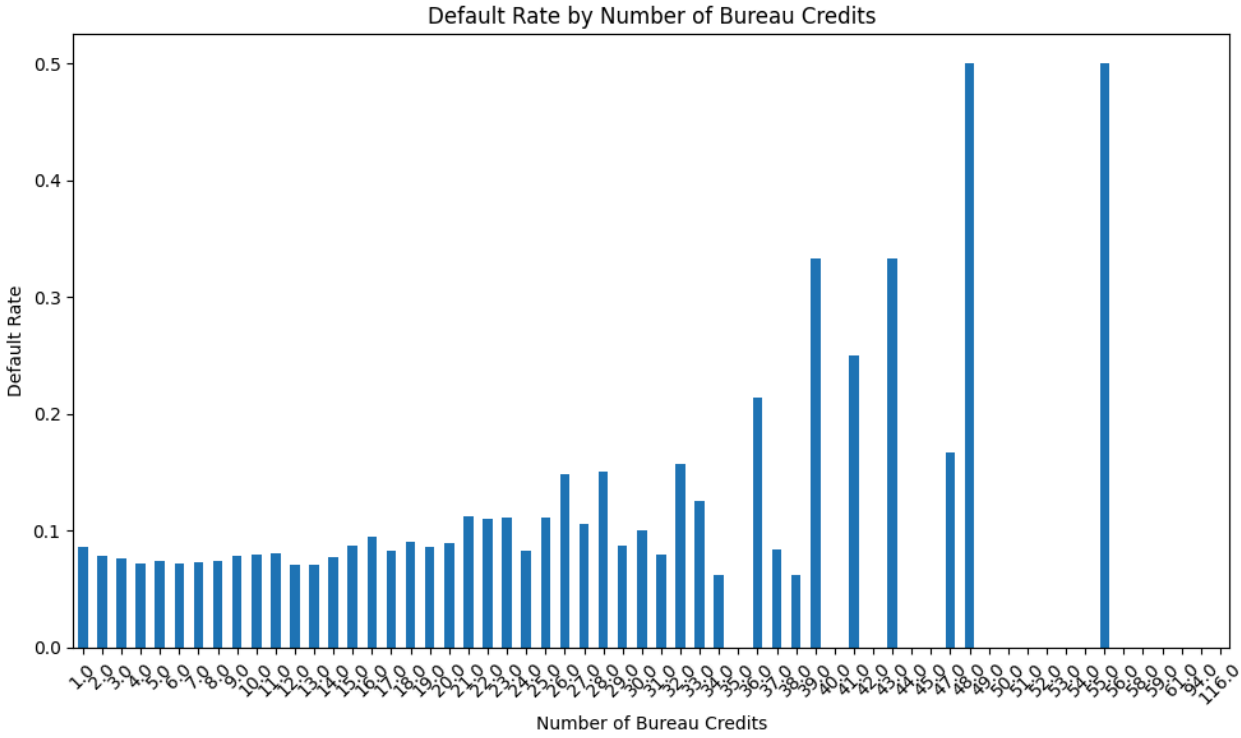
Key Observations:

- There is **no clear trend** that more bureau credits lead to higher default rates.
- Instead, **very high numbers of bureau credits (e.g., 35, 48, 55, 58, 61, 94, 116)** are associated with **extremely high or 100% default rates**, which may suggest **data anomalies, errors, or a possible outlier effect** (e.g., fraudulent applications or data entry issues).
- Some applicants with **zero or low bureau credits** have **zero default rates**, while others with **moderate credits** (e.g., 10–20) have default rates around **7–8%**, which is relatively low.

Conclusion:

Applicants with **very high numbers of bureau credits** (e.g., 50+) are **not** consistently at higher risk — some show **zero defaults**, others show **very high defaults**. This suggests that **credit count alone is not a reliable predictor** of default risk. Instead, **the distribution of bureau credits appears to contain outliers or data issues** that may need further investigation.

Thus, **more bureau credits do not reliably correlate with higher default rates** — the relationship is complex and potentially influenced by data quality or other underlying factors.



View generated code

```
import pandas as pd
import matplotlib.pyplot as plt

# Load the necessary datasets
app_train = pd.read_csv('data/application_train.csv')
bureau = pd.read_csv('data/bureau.csv')

# Aggregate bureau data to count the number of bureau credits per cli
bureau_agg = bureau.groupby('SK_ID_CURR')['SK_ID_BUREAU'].count().res
bureau_agg.columns = ['SK_ID_CURR', 'NUM_BUREAU_CREDITS']

# Merge the aggregated bureau data back to the application train data
df = app_train.merge(bureau_agg, on='SK_ID_CURR', how='left')

# Calculate the default rate for each group of bureau credits
default_rate = df.groupby('NUM_BUREAU_CREDITS')['TARGET'].mean()

# Plot the results
plt.figure(figsize=(10, 6))
default_rate.plot(kind='bar')
plt.title('Default Rate by Number of Bureau Credits')
plt.xlabel('Number of Bureau Credits')
plt.ylabel('Default Rate')
```

```
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

# Print the default rates
print(default_rate)
```

What percentage of applicants have active credits in other institutions?

36.74% of applicants have active credits in other institutions. This indicates that a significant portion of borrowers maintain existing credit relationships with other lenders, which could imply higher credit risk due to potential financial strain from multiple obligations. Monitoring this metric helps assess the overall credit health and default risk of applicants.

View generated code

```
import pandas as pd

# Load the necessary data
bureau = pd.read_csv('data/bureau.csv')

# Filter for active credits
active_credits = bureau[bureau['CREDIT_ACTIVE'] == 'Active']

# Calculate the percentage of active credits
percentage_active_credits = (len(active_credits) / len(bureau)) * 100

# Print the result
print(f"Percentage of applicants with active credits in other institu
```

What is the default rate of applicants who previously had overdue bureau loans?

The default rate of applicants who previously had overdue bureau loans is **15.90%**. This indicates that among borrowers with a history of overdue loans in the bureau records, approximately 15.9% defaulted on their loans. This is a significant risk indicator, suggesting

that such applicants are more likely to default compared to those without such a history, and thus may require more stringent credit evaluation or higher interest rates.

View generated code

```
import pandas as pd

# Load necessary datasets
app_train = pd.read_csv('data/application_train.csv')
bureau = pd.read_csv('data/bureau.csv')

# Filter out applications with any overdue bureau loans
overdue_bureaus = bureau[bureau['CREDIT_DAY_OVERDUE'] > 0]['SK_ID_CUR

# Merge with application train to get target values
df = app_train[app_train['SK_ID_CURR'].isin(overdue_bureaus)]

# Calculate the default rate
default_rate = df['TARGET'].mean()

print(f"Default rate of applicants who previously had overdue bureau
```

What factors are most associated with loan default?

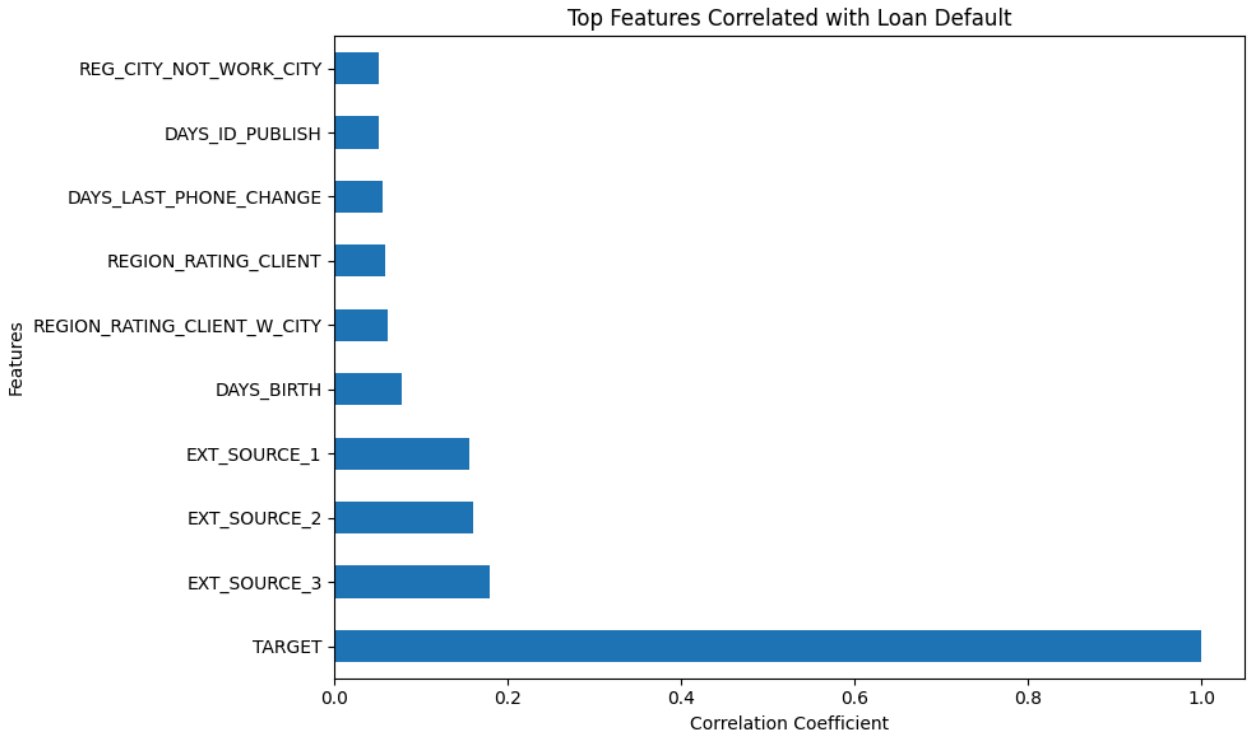
The factor most associated with loan default is **EXT_SOURCE_3**, with a correlation coefficient of **0.1789**, followed by **EXT_SOURCE_2** (0.1605) and **EXT_SOURCE_1** (0.1553). These external credit scoring sources appear to carry the strongest statistical relationship with the target variable (loan default).

Other notable factors include:

- **DAYS_BIRTH** (correlation 0.0782): Age of the borrower, suggesting younger borrowers may be at higher risk.
- **REGION_RATING_CLIENT_W_CITY** and **REGION_RATING_CLIENT** (both ~0.06): Regional ratings may influence default risk, possibly due to economic conditions or access to credit.
- **DAYS_LAST_PHONE_CHANGE** (0.0552): Indicates that changes in phone usage may reflect changes in stability or income.

While these correlations suggest associations, they do not imply causation. The strongest predictors are the external sources (EXT_SOURCE_1–3), which likely reflect third-party credit assessments or behavioral data. These could be key indicators for modeling default risk in practice.

(Note: The chart generated in the code would visually represent these correlations, showing the strength and direction of each relationship.)



View generated code

```
import pandas as pd
import matplotlib.pyplot as plt

# Load the necessary datasets
app_train = pd.read_csv('data/application_train.csv')

# Select numeric columns and calculate correlations
numeric_cols = app_train.select_dtypes(include='number').columns
correlations = app_train[numeric_cols].corr()

# Find the top 10 features most correlated with TARGET
top_features = correlations['TARGET'].abs().sort_values(ascending=Fal

# Print the top features and their correlation coefficients
print(top_features)

# Plotting the top features
```

```
plt.figure(figsize=(10, 6))
top_features.plot(kind='barh')
plt.title('Top Features Correlated with Loan Default')
plt.xlabel('Correlation Coefficient')
plt.ylabel('Features')
plt.tight_layout()
```

Ask a question about the Home Credit dataset...